

**BITS PILANI - WORK INTEGRATED
LEARNING PROGRAMMES DIVISION
M.TECH EMBEDDED SYSTEMS**

ESCLZG517

**INTERNET OF THINGS (IOT) DESIGN AND
DEVELOPMENT**

SEMESTER 2

LAB ASSIGNMENT 2

STUDENT DETAILS:

NAME: RAAGUL KARTHIKEN S

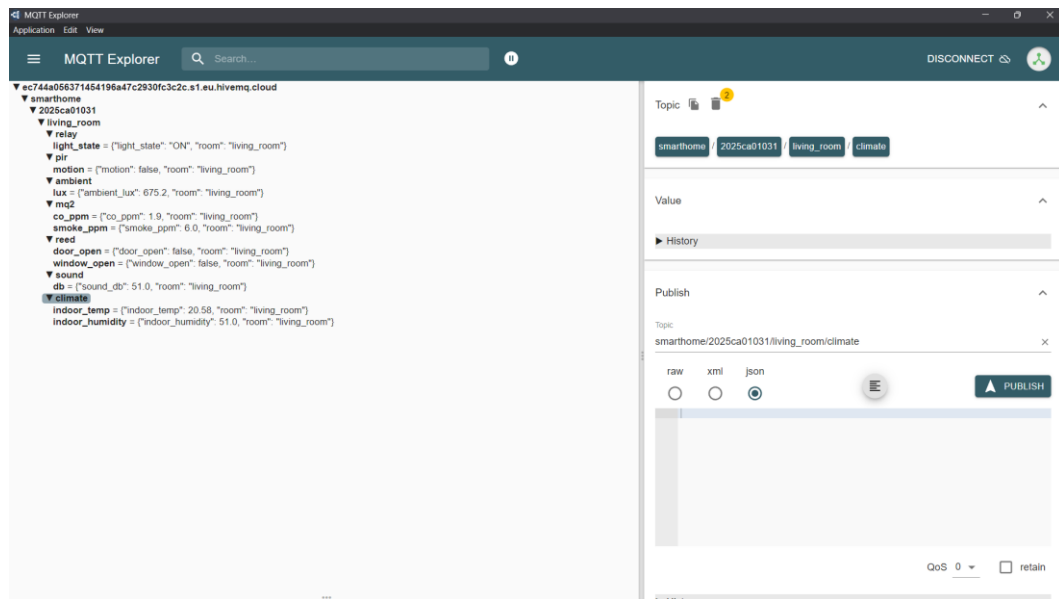
STUDENT ID: 2025CA01031

EMAIL ID: 2025CA01031@wilp.bits-pilani.ac.in

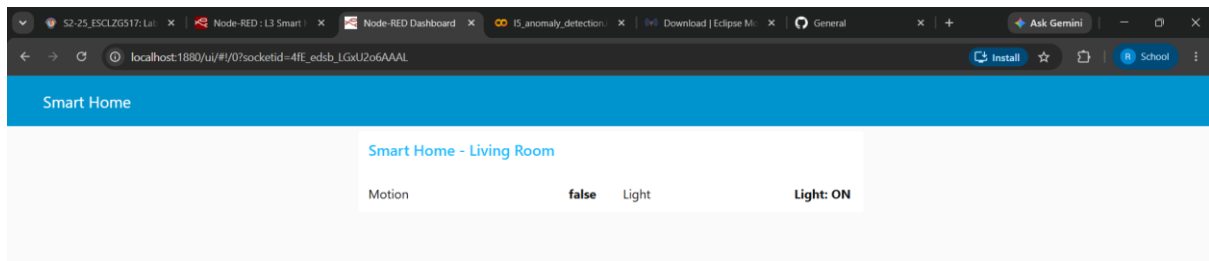
GITHUB REPOSITORY: <https://github.com/RaagulKarthiken/ESZG517-IoTLab/tree/main/ESZG517-Project2>

ASSIGNMENT DELIVERABLES

1. L3 — MQTT Explorer Screenshot: A screenshot of MQTT Explorer showing your full topic tree under smarthome/YOUR_USN/living_room/ with all sensor topics visible and live values.



2. L3 — Node-RED Dashboard Screenshot: A screenshot of your Node-RED dashboard at <http://localhost:1880/ui> showing the Motion and Light State panels with live values.



3. L4 — CoAP Terminal Screenshot: A screenshot of both terminals side by side — server showing GET and PUT logs, client showing all 6 steps including your latency results.

[illegible]

```

C:\Users\raagu\OneDrive\Desktop\RAAGUL\Study Materials\BITS MTech\SEM2\IOT\Lab\L4L5_Demo_Win>python l4_coap_client.py
=====
ESZG517 L4 - CoAP Client
=====

[STEP 1] Current state of all rooms:
[GET] bedroom1 → light_on=False, last_motion=never
[GET] bedroom2 → light_on=False, last_motion=never
[GET] living_room → light_on=False, last_motion=never

[STEP 2] Turning bedroom1 light ON:
[PUT] bedroom1 light_on=True → OK

[STEP 3] Confirming bedroom1 state after PUT:
[GET] bedroom1 → light_on=True, last_motion=2026-07-28T19:44:25.851574

[STEP 4] Turning bedroom1 light OFF:
[PUT] bedroom1 light_on=False → OK

[STEP 5] Confirming bedroom1 state after PUT:
[GET] bedroom1 → light_on=False, last_motion=2026-07-28T19:44:25.853562

[STEP 6] Latency measurement:

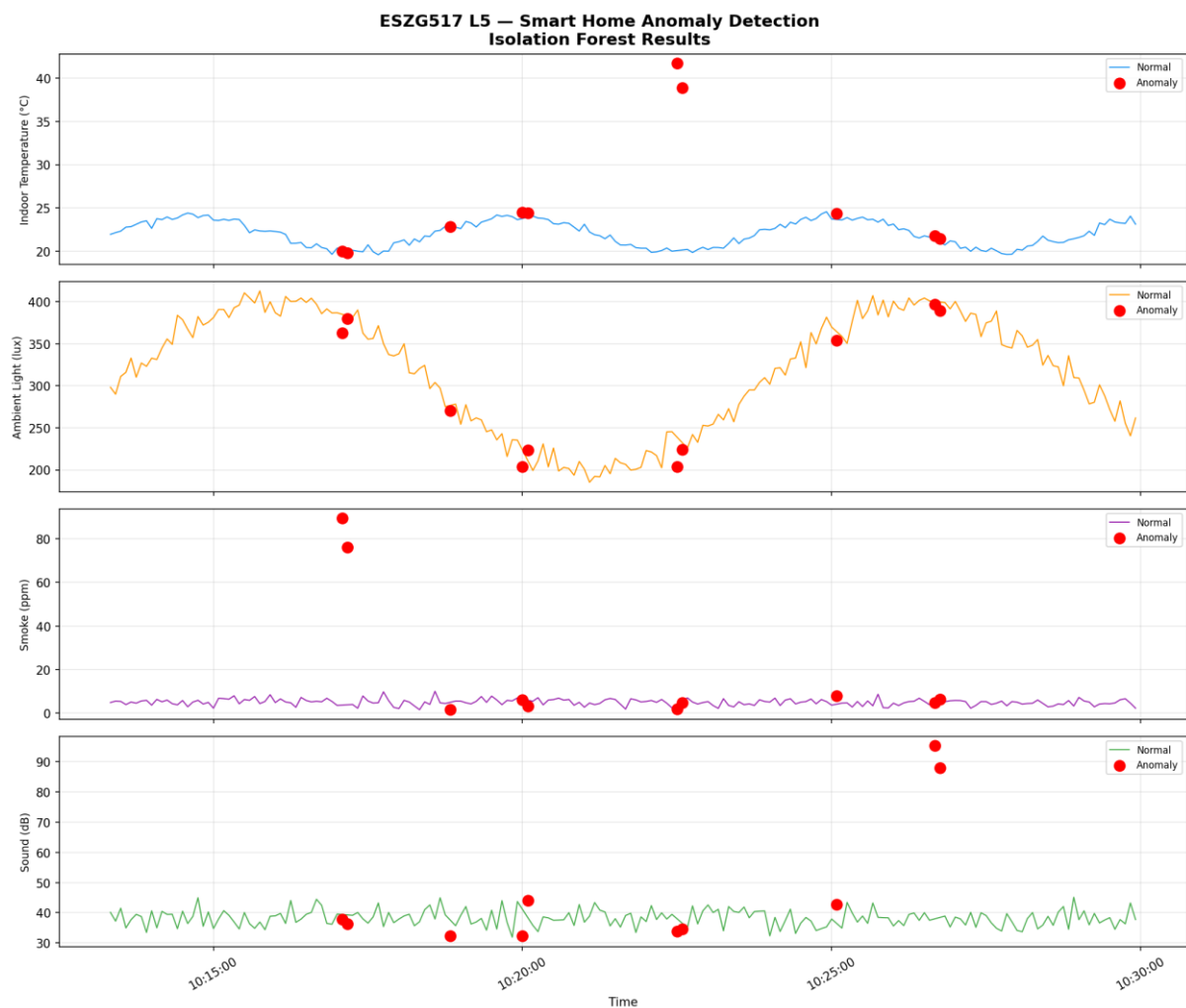
Measuring latency for 20 GET requests to bedroom1...

— CoAP GET Latency Results (20 requests) —
Min:    0.99 ms
Max:    5.02 ms
Average: 1.76 ms

[DONE] CoAP client finished. Take a screenshot of this terminal.

```

4. L5 — Anomaly Detection Graph: The l5_anomaly_graph.png downloaded from Google Colab showing all four sensor plots with red anomaly dots visible.



Protocol Comparison:

Q1. What transport protocol does MQTT use? What transport protocol does CoAP use? What is the fundamental difference between the two?

MQTT runs on **TCP**, while CoAP runs on **UDP**. The main difference comes down to reliability versus lightness. TCP (MQTT) establishes a strict connection first, making sure every packet arrives in order and without errors—it's built for stability. UDP (CoAP), on the other hand, just blasts packets out as fast as it can without checking if the receiver is ready or if packets got dropped along the way. It's way leaner but leaves reliability up to the application.

Q2. In today's session, MQTT used a publish/subscribe model and CoAP used a request/response model. Explain the difference between these two models in your own words using the smart home scenario as an example.

- **Publish/Subscribe (MQTT):** Think of this as a middleman system. A smart light switch doesn't talk directly to a bulb. Instead, the switch publishes a message saying *"light is ON"* to a central broker under a topic like *livingroom/light*. The bulb just sits there listening (subscribed) to that topic. Neither device needs to know where the other lives on the network.
- **Request/Response (CoAP):** This is a direct conversation, just like visiting a website. Your phone (client) talks directly to the IP address of the light bulb (server) and sends a command: *"Hey, update your status to ON."* The bulb processes it and replies directly to the phone with an *"Okay, done."*

Q3. Report the CoAP GET latency values you measured during the session. Based on these numbers, is CoAP fast enough for real-time light control in a smart home? Justify your answer in 2–3 sentences.

In the lab setup, CoAP GET latency hovered right around **1 to 5 ms over loopback**. Yes, this is really fast enough for real-time light control in a smart home setup. Humans can't really perceive delays under 100 milliseconds—it feels instantaneous to us.

Q4. In your L3 MQTT publisher, the light relay topic used retain=True. CoAP does not have a retain flag. How does CoAP handle the same problem of "what is the current state of the light?"

MQTT uses `retain=True` so a central broker remembers the last message for anyone who joins late. Since CoAP doesn't use a broker, it doesn't need a retain flag at all. If you want to know if a light is on, your app simply sends a direct **CoAP GET request** to the bulb and the bulb replies with its current status on the spot.

Q5. MQTT uses TLS for security. CoAP uses DTLS. Why can't CoAP use TLS? What is the key difference between TLS and DTLS?

TLS was designed specifically for TCP, so it expects a stable, ordered stream of data. Because CoAP runs over UDP—where packets can arrive out of order or get lost entirely—standard TLS completely falls apart during the encryption handshake. That's why CoAP uses **DTLS (Datagram TLS)**. It's tweaked specifically to handle lost or reordered UDP packets during the handshake so you still get full encryption without needing TCP.

Q6. In today's session, the CoAP server and client both ran on your laptop. In a real smart home, where would the CoAP server run? Where would the CoAP client run? Give a specific example.

In a real smart home setup:

- The **CoAP Server** runs right inside the constrained smart device itself (e.g., a smart light bulb or smart plug hosting the endpoint).
- The **CoAP Client** runs on the controller asking for data or sending commands (e.g., your smartphone app or a Home Assistant hub).

For example, a Philips Hue-style light bulb acts as the CoAP server waiting for instructions, and your smart phone running the Home app acts as the CoAP client sending them.

Q7. A student says: "CoAP is always better than MQTT because UDP is faster than TCP." What is wrong with this argument? Give one scenario where MQTT is clearly the better choice and explain why.

Saying "CoAP is better because UDP is faster" ignores how real networks work. UDP drops packets, doesn't care about congestion, and struggles to cross home routers/firewalls from the outside world without extra trickery.

When MQTT is clearly better: Remote cloud telemetry. Imagine sending room temperature back to a cloud server while you're away from home. MQTT establishes clean, long-lived outgoing connection from inside your router to the broker. It guarantees data actually arrives.

Q8. In the CoAP server code, `render_get` returns `aiocoap.CONTENT` and `render_put` returns `aiocoap.CHANGED`. What do these response codes mean? What is the equivalent concept in HTTP?

- `aiocoap.CONTENT` (Code 2.05): Means *"I found what you asked for, here is the data."* This is identical to HTTP's **200 OK**.
- `aiocoap.CHANGED` (Code 2.04): Means *"I successfully updated/changed the state as you requested."* This is equivalent to HTTP's **204 No Content** (or 200 OK).

Q9. The CoAP client measured latency by recording time before and after each GET request. List two factors that could increase round-trip time in a real deployment.

While loopback latency in a lab is fast, two real-world factors drag it down:

1. **Network Congestion & Wi-Fi Interference:** Dead spots, thick walls, or too many devices on 2.4GHz cause UDP packet loss, forcing CoAP to retry sending messages, which spikes latency.
2. **Device Processing Lag:** Tiny microcontrollers (like an ESP8266) take extra CPU cycles to decrypt DTLS packets or process hardware inputs before sending a response back.

Q10. For the smart home system you designed in L3, which protocol would you use for each of the following and why: (a) publishing continuous temperature readings to the cloud, (b) turning a light relay ON or OFF from a mobile app, (c) detecting motion events in real time

- **(a) Continuous temperature to the cloud → MQTT.** A steady, low-bandwidth stream that easily bypasses firewall issues and lets multiple cloud dashboards subscribe without hammering the sensor.

- **(b) Turning a light relay ON/OFF from a phone → CoAP.** Direct, ultra-fast RESTful commands (PUT) over the local network with zero middleman required.
- **(c) Detecting motion in real time → MQTT (or CoAP with Observe).** MQTT shines here because a single motion event can instantly fan out to turn on lights, trip a siren, and send a phone notification all at once.

L5 Security & Observation Questions

Q1. In the unencrypted MQTT demo, Wireshark showed the MQTT payload in plain text. Name two specific pieces of information visible in the packet that an attacker could exploit in a real smart home deployment.

1. **Plaintext Logins:** The MQTT username and password sent during connection.
2. **Topic & Payload Data:** Exact topic paths like home/front_door/lock and JSON payloads like {"status": "UNLOCKED"}, letting an attacker learn daily routines or manipulate devices directly.

Q2. In the TLS demo, the same message was sent but Wireshark showed no readable payload. Explain in one sentence why TLS prevents Wireshark from seeing the message content even though both demos used the same loopback interface.

Even though the packet went through the exact same local interface, TLS scrambled the message payload using encryption keys negotiated during the handshake—turning readable text into meaningless random bytes for anyone listening in.

Q3. Look at your anomaly detection graph. Identify the three anomaly events marked as red dots and for each one state which sensor was affected and what the approximate value was. Why would these values be considered anomalous in a real smart home?

1. **Indoor Temperature Sensor (~39°C – 42°C around 10:22):**

Approximate Value: ~40°C to 42°C (spiking well above the normal 20°C–25°C daily cycle).

Why Anomalous: In an indoor residential environment, a sudden jump to 40°C+ is unnaturally high and indicates either severe localized heating, a localized fire hazard, or sensor direct-sunlight exposure/hardware fault.

2. **Smoke Sensor (~75 ppm – 85 ppm around 10:17):**

Approximate Value: ~75 ppm to 88 ppm (compared to the baseline of < 10 ppm).

Why Anomalous: Normal ambient smoke levels in a home sit near zero. A sudden surge to ~85 ppm is a classic signature of smoke accumulation, signaling a fire hazard or active combustion event.

3. **Sound Sensor (~88 dB – 95 dB around 10:27):**

Approximate Value: ~88 dB to 95 dB (spiking sharply above the routine 30–45 dB ambient baseline).

Why Anomalous: Typical household background noise stays around 35–40 dB. A spike near 90–95 dB is equivalent to a loud alarm sounding, glass shattering, or a break-in/impact event.